

Reliability Tiers for AI-Assisted Mathematics: A Verification-Centred Framework, Failure Taxonomy, and Audit Protocol

Challenge C2 Submission*

September 30, 2026

Abstract

Large language models increasingly produce mathematical arguments that read like proofs but fail under scrutiny. The research community has responded with three families of remedies: sampling and self-consistency, learned verifiers with process supervision, and formal theorem proving. Practitioners still lack a way to state what a given artefact has actually been checked for. This paper makes that question explicit. We contribute (i) a ten-item failure taxonomy for AI-generated mathematical claims (F1–F10) that separates statement-level defects from inference-level, numeric, attributional, and reproducibility defects; (ii) a six-level verification tier framework (T0–T5) in which each tier is characterised by the guarantee it can and cannot provide, its cost class, and the defects it retires; (iii) two propositions explaining why tiers are monotone and why no sound and complete computable verifier for arithmetic exists, which forces any usable framework to report residual risk rather than certify truth; and (iv) a twelve-item audit protocol with a claim-level provenance ledger. The central practical claim is that formal verification retires inference-soundness defects but cannot retire statement-fidelity or provenance defects, so a reliable AI4Math pipeline must pair a formal tier with an independent adversarial audit tier. We state the boundaries of the framework explicitly: it is a literature-grounded analytic synthesis, and its cost classes are order-of-magnitude estimates rather than measured timings. Where measurement is possible we supply it: the source compiles under a pinned CI toolchain with no undefined references and the build log is committed with the paper, and a hand-built probe of the fidelity gate catches 14 of 16 injected statement drifts with zero false alarms on 6 faithful controls, while exposing a blind spot for converse swaps.

1 Introduction

Mathematical reasoning was long treated as a domain where machine output could be checked cheaply: a result is either proved or it is not. Recent systems have made that intuition feel achievable. AlphaGeometry solves olympiad-level geometry problems without human demonstration (Trinh et al., 2024), FunSearch discovers new mathematical constructions by searching over programs (Romera-Paredes et al., 2024), and neural provers interacting with the *Lean* proof assistant make measurable progress on formal benchmarks (Zheng et al., 2021; Yang et al., 2023; Lin et al., 2024).

The same literature documents a persistent reliability gap. Language models score highly on mathematical word problems (Cobbe et al., 2021; Hendrycks et al., 2021) while remaining

*Prepared as a submission for Challenge C2 (AI for Math). The companion AI log records the literature search, every machine verification step, and every AI-generated claim that was checked by hand.

brittle in ways that matter: performance degrades when a problem statement gains a semantically irrelevant clause (Mirzadeh et al., 2024), intrinsic self-correction can lower accuracy rather than raise it (Huang et al., 2023), models assert incorrect numerical relationships with confidence, and generated bibliographies contain fabricated entries (Ji et al., 2023). A reader who encounters a fluent paragraph labelled “proof” therefore learns almost nothing about its epistemic status.

Our starting observation is that “a proof” denotes at least five different things in current practice: (a) a fluent informal argument, (b) an argument that several independent samples agree on, (c) an argument whose steps passed a learned or human step-level review, (d) a term accepted by a proof assistant kernel, and (e) a claim reproduced by an independent party. These are not interchangeable, yet downstream consumers – reviewers, students, engineering teams – tend to read them as one. The consequence is systematic over-trust: a claim certified at a weak level is quoted as if it had been certified at a strong one.

This paper argues that the fix is not a better single tool but an explicit accounting of verification level. We contribute:

1. **A failure taxonomy (F1–F10).** Ten defect classes for AI-generated mathematical claims, organised by the layer at which the defect enters: statement, inference, numeric, attribution, or process (Section 3).
2. **A tier framework (T0–T5).** A partial order on verification levels, each carrying a stated guarantee, a cost class, and a coverage profile over the taxonomy (Section 4).
3. **Two propositions and their consequences.** Tiers are monotone by construction (Prop. 1); and by Gödel’s incompleteness theorem no sound and complete computable verifier for arithmetic exists (Prop. 2), so residual risk is a structural feature of the problem rather than an implementation defect (Section 5).
4. **An audit protocol.** A twelve-item checklist and a claim-level provenance ledger that make the tier of every claim auditable after the fact (Section 6).

The framework’s central operational claim is asymmetric and easy to get wrong: formal verification retires inference-soundness defects but *cannot* retire statement-fidelity defects, because the fidelity of a formal statement to an informal intention is itself an informal judgement. Likewise no prover kernel can detect a fabricated citation or a contaminated benchmark. A pipeline that stops at the formal tier therefore remains unreliable in exactly the ways that matter for published mathematics. Section 7 states where the framework is weak and what evidence we did not collect.

2 Related Work

2.1 Informal mathematical reasoning with language models

Benchmark progress is the visible surface of the field. GSM8K established grade-school word problems as a reasoning test bed (Cobbe et al., 2021), and MATH raised the difficulty to competition level (Hendrycks et al., 2021). Chain-of-thought prompting and self-consistency – sampling several reasoning paths and taking a majority answer – produced large gains and remain the default baseline (Wang et al., 2022). These methods optimise the probability of a correct final answer. They provide no guarantee about the argument that produced it, and a majority answer is evidence about a model’s distribution, not about the truth of a statement.

2.2 Learned verifiers and process supervision

A second family replaces sampling agreement with a learned critic. Outcome-based reward models score final answers; process supervision scores intermediate steps, and step-level supervision of a verifier was found to outperform outcome supervision for the purpose of selecting correct solutions (Lightman et al., 2023). Uesato et al. compare process-based and outcome-based feedback for word problems (Uesato et al., 2022), and Math-Shepherd removes the need for human step labels by constructing them automatically (Wang et al., 2023). Self-correction completes the picture only when it is driven by an external signal: a learned corrector that consumes such a signal improves generation (Welleck et al., 2022), whereas intrinsic self-correction without external feedback does not reliably improve reasoning and can degrade it (Huang et al., 2023). Models have partial access to their own error signal (Kadavath et al., 2022), but calibration is imperfect, which is why we treat self-reported confidence as weak evidence and build no tier on it.

The epistemic status of this family is statistical. A verifier that is 90% accurate across a distribution of steps does not certify that *this* step is valid; it changes the odds. Its failure mode is therefore correlated with the generator’s: both may share the same misconception, and neither can detect it.

2.3 Formal theorem proving and autoformalisation

The third family produces machine-checkable certificates. Proof search over formal libraries began with language models trained on proof steps (Polu and Sutskever, 2020) and was extended by tree search over proof states (Lample et al., 2022), retrieval over large libraries (Yang et al., 2023), and interleaving of informal reasoning with tactic execution (Lin et al., 2024); DeepSeek-Prover shows that large-scale synthetic data alone moves the formal benchmark frontier (Xin et al., 2024). Benchmarks such as MiniF2F (Zheng et al., 2021) and PutnamBench (Tsoukalas et al., 2024) make these claims comparable across systems, and Lean 4 provides a modern kernel and tactic language (de Moura and Ulrich, 2021).

Autoformalisation is both the bridge and the weak point. Wu et al. translate natural-language competition problems into formal statements (Wu et al., 2022), and draft–sketch–prove decomposes an informal proof into formal sketches and steps (Jiang et al., 2022). In both settings the system proves a *formal* sentence; whether that sentence is the one the author meant is decided outside the kernel. At the other end of the scale, AlphaGeometry closes geometry problems with a symbolic engine (Trinh et al., 2024), FunSearch searches over programs and produces new constructions (Romera-Paredes et al., 2024), and the Flyspeck project shows what large-scale formalisation really costs: a decade-long effort with twenty-two listed authors to formalise a single theorem (Hales et al., 2015).

2.4 Critiques, brittleness, and provenance

Evidence of brittleness is now systematic. Adding a semantically irrelevant clause to a word problem can move reported accuracy substantially across model families (Mirzadeh et al., 2024); the mathematical behaviour of ChatGPT on well-defined tasks was uneven from the start (Frieder et al., 2023); and generated text, including its bibliographic apparatus, can be fluent and false (Ji et al., 2023). Automated research loops that write, review, and iterate without a human in the loop are now attempted (Lu et al., 2024), which raises rather than lowers the value of an audit trail. Architectures that pair a generator with external critics – LLM-Modulo – are the closest existing relative of the tier idea (Kambhampati et al., 2024); retrieval grounding reduces, but does not eliminate, unsupported assertions (Lewis et al., 2020).

Table 1: A failure taxonomy for AI-generated mathematical claims. “Retired at” names the lowest tier whose checks can in principle detect and reject the defect; in practice a check may miss an instance, which is what the residual set records.

ID	Defect	Layer	Minimal illustration	Retired at
F1	Statement drift (misformalisation)	statement	A formal statement silently drops a side condition, so a different theorem is proved	T5
F2	Quantifier or converse error	statement	Proving “continuous implies differentiable” while reporting the converse	T4/T5
F3	Unsound inference step	inference	A step interchanges limits without justification	T4
F4	Undischarged assumption	inference	A lemma is used outside the hypotheses under which it was proved	T4
F5	Vacuity or triviality	inference	The hypotheses are unsatisfiable, so the theorem has no content	T5
F6	Numeric or monotonicity slip	numeric	Asserting $\pi > 22/7$, contradicted by Archimedes’ bound	T2
F7	Citation fabrication or misattribution	attribution	A bibliographic entry whose title or identifier does not exist	T5
F8	Contamination or leakage	attribution	A benchmark result obtained on problems present in training data	T5
F9	Over-generalisation	process	A general law asserted from a handful of examples	T3/T5
F10	Irreproducible artefact	process	Results shipped without a pinned environment or a runnable pipeline	T5

What is missing from this literature is a shared currency for *how much* was verified. Papers report pass rates at whatever tier they happen to use and leave the transfer question implicit. Our framework supplies the currency and, more importantly, states what each unit of currency cannot buy.

3 A Failure Taxonomy for AI-Generated Mathematical Claims

Before assigning levels we need a vocabulary of what can go wrong. We organise defects by the layer at which they enter a claim: the *statement* (what is being asserted), the *inference* (how it is derived), the *numeric* layer (concrete quantitative assertions), the *attribution* layer (what the claim is attached to), and the *process* layer (how the result was produced and reported). Table 1 lists ten classes.

Two design choices matter. First, we separate F1 from F3 deliberately: a derivation that is internally flawless but establishes the wrong sentence is not a proof of the intended claim, and no formal pipeline can see the difference. Second, F7 and F8 belong to the attribution layer rather than the inference layer, because they are falsehoods *about* the work rather than errors *in* it; they survive every kernel and every verifier.

4 A Tiered Verification Framework

4.1 Claims, evidence, and certification

Definition 1 (Claim). A claim is a tuple $c = (S, A, P, E)$, where S is the statement (informal, formal, or both), A is the set of assumptions under which S is asserted, P is the derivation artefact (a prose proof, a proof script, or a program), and E is the evidence bundle, that is, the records produced by the checks that were run.

Definition 2 (Tier predicate). For $t \in \{0, \dots, 5\}$, let V_t be the predicate “ c passes every check of tier t ”. The checks are listed in Table 2; the check set of tier t contains the check sets of all lower tiers.

Definition 3 (Certification). c is *certified at level t* , written $\text{tier}(c) = t$, if t is the largest index such that $V_t(c)$ holds. Certification is always relative to a tier: we never write that a claim is verified, only that it is certified at level t .

Definition 4 (Coverage and residual risk). $\Delta(t) \subseteq \{F1, \dots, F10\}$ denotes the defect classes that the checks of levels at most t can in principle detect. The residual risk of a claim certified at level t is $R(c) = \{F1, \dots, F10\} \setminus \Delta(t)$.

Proposition 1 (Monotonicity). *If $V_t(c)$ holds, then $V_{t'}(c)$ holds for every $t' \leq t$.*

Proof. Immediate from Definition 2 by construction of the check sets. We state it because it is routinely violated in practice: an artefact is described as “formally verified” on the strength of the inference layer (F3–F4) while its statement fidelity (F1) was never reviewed. $\square$

4.2 Assigning a tier

The procedure below is what a practitioner actually runs. It is written to fail closed: a step that cannot be completed lowers the tier rather than being skipped.

1. **Inventory.** Enumerate every claim the artefact makes, including claims in tables, captions, abstracts, and code comments.
2. **Statement ledger.** Record S and A verbatim together with the intended informal meaning expressed in the author’s own words.
3. **Numeric pass.** Identify quantitative sub-claims and test each with an independent tool; Section 5 gives a case in which this alone refutes a plausible claim.
4. **Inference pass.** Attempt formalisation of each load-bearing claim. A term accepted by a kernel certifies the inference layer. Failure of formalisation is evidence about the derivation, not about the statement.
5. **Fidelity pass.** A party that did not write the formal statement – human or otherwise – compares S against the recorded intent. This gate is what retires F1 and F2.
6. **Adversarial pass.** Attempt refutation rather than confirmation: search for counterexamples, verify every citation against a primary source, and check for contamination.
7. **Label and disclose.** Publish the tier, the residual set $R(c)$, and the evidence bundle.

Steps 3 and 4 are mechanical and largely automatable. Steps 2, 5, and 6 are judgement-bearing and must be assigned to an accountable party. The most common pipeline defect we observe is stopping after step 4 and calling the result verified.

Table 2: The tier framework. Costs are order-of-magnitude classes relative to producing the claim once: $1\times$ (no extra pass), $O(10)$, $O(10^2)$, $O(10^3)$ or more. The rightmost column is the part of the guarantee that the tier does *not* provide.

Tier	Check performed	Guarantee provided	Cost	Defects retired	Cannot retire
T0	Generation only (prose answer, no independent check)	None; fluency is not evidence	$1\times$	—	F1–F10
T1	Statistical agreement: repeated sampling, voting, outcome reward models	Confidence in the answer under the model’s own distribution	$O(10)$	partial F6	F1–F5, F7–F10
T2	Mechanical numeric or symbolic check (rational arithmetic, computer algebra, counterexample search)	Refutation of concrete quantitative sub-claims	$O(1)$ per check	F6	F1–F5, F7–F10
T3	Structured step review with explicit lemma dependencies (process reward model or human reading)	Local plausibility of each step under stated hypotheses	$O(10^2)$	F3, F9 (partial)	F1, F2, F5, F7, F8, F10
T4	Formal verification: statement and proof accepted by a kernel	Derivability of the formal sentence from the formal hypotheses	$O(10^3)$	F3, F4	F1, F2 (detection), F5, F7–F10
T5	Independent adversarial audit: fidelity review, citation and provenance check, contamination check, reproduction from a pinned environment	Detection of statement, attribution, and process defects	human-dependent	F1, F2, F5, F7, F8, F10	unintended defects outside the audited set

5 Analysis

5.1 Why the formal tier cannot retire statement fidelity

Remark 1 (Argument, not a theorem). $\Delta(4) \cap \{F1, F2\} = \emptyset$: the checks available up to tier 4 cannot detect statement drift or a quantifier error.

We state this as an argument rather than a theorem, and give the argument explicitly. A kernel checks a derivation relation between a formal sentence and a formal proof term. Fidelity, by contrast, is a relation between an informal intention and a formal sentence, and the only evidence available for it is informal. Three consequences follow. First, a successful kernel check is consistent with having proved a drifted statement, and a failed check is consistent with having mis-formalised a true one; the kernel is silent about which case obtains. Second, additional prover effort does not close the gap, because the object that would have to be checked – the intention – is not a formal object. Third, formalising “the intended meaning” cannot discharge the obligation either: it relocates the judgement to whoever writes and reviews the specification, without removing it. A fully formal specification of intention would refute this remark; for open mathematical claims we know of none.

Table 3: Probe of the mechanical fidelity check (`python run-probe.py`). Proportions describe this checker on this hand-built probe set, not a population.

Quantity	Value
pairs / drifted / faithful controls	22 / 16 / 6
injected drifts detected	14
coverage on injected drifts	14/16 = 0.875
coverage, F1 class (surface changes)	10/10 = 1.000
coverage, F2 class	4/6 = 0.667
false alarms on faithful controls	0/6 = 0.000
precision when an alarm is raised	14/14 = 1.000
defect class correct when flagged	14/14 = 1.000

5.2 A reproducible probe of the fidelity gate

The previous subsection argues that the fidelity gate cannot be replaced by formal checks. It does not follow that the gate is unmechanisable. To fix how far mechanisation goes, we built a minimal checker, ran it on a hand-built probe set, and put both in the artefact (`c2-f1-probe/`); the numbers below are reproduced in continuous integration on every change.

The checker reads four surface features from a statement – condition atoms drawn from a 24-word table (`nonzero`, `positive`, `continuous`, `bounded`, `compact`, `prime`, ...), the quantifier class (universal or existential), the numerals occurring in the sentence, and the count of strict inequalities – and compares an intent statement against its formalisation. A differing quantifier class is reported as F2, any other difference as F1, and no difference raises no alarm. The probe set holds 22 (intent, formalisation) pairs: 16 injected drifts across four surface classes (missing side condition 4, changed bound 3, changed strictness 3, quantifier misplacement 4) plus 2 converse or direction swaps that are surface-identical by construction, against 6 faithful controls. Ground-truth labels were fixed before the checker ran.

Two findings matter for the framework. First, the whole surface-visible class of F1 is mechanisable at negligible precision cost: all ten cases of missing side conditions, changed bounds, and changed strictness were flagged, with zero false alarms on the controls, so the check can run in a pipeline without adding review noise. Second, the F2 class split. Quantifier misplacement was caught in all four cases, but both converse and direction swaps were missed, and the reason is structural rather than an implementation gap: “differentiable implies continuous” and “continuous implies differentiable” differ in no atom, no numeral, no quantifier class, and no inequality count. This is the empirical form of the argument above – no check over the sentence alone can separate a claim from its converse – which is why the gate must still be signed by a party that did not write the formalisation.

The same standard applies to the artefact itself. `paper.tex` is compiled in continuous integration on every change to the source or the bibliography, and the PDF, the full L^AT_EX log, and a machine-generated evidence file are committed under `build/`, so the claim that the paper compiles can be checked without trusting the authors.

5.3 Three worked examples

Example 1 (F1, invisible to the kernel). The intended claim is that for every odd prime p the integer $p^2 - 1$ is divisible by 8. A pipeline formalises the side condition away:

$$\forall p \in \mathbb{N}, \quad \text{Prime}(p) \implies 8 \mid (p^2 - 1).$$

This sentence is false: $p = 2$ gives $p^2 - 1 = 3$ and $8 \nmid 3$. The prover fails, and the failure carries no information about the intended claim. If instead the pipeline silently strengthens the hypothesis until something provable appears, the artefact ships a theorem no reader asked for. Both paths pass through the kernel without touching the defect. The intended claim is true, and its one-line reason is worth recording precisely because it shows how much content a drift can hide: for odd p , the integers $p - 1$ and $p + 1$ are consecutive even numbers, so one is divisible by 4 and the other by 2, hence $8 \mid (p - 1)(p + 1) = p^2 - 1$.

Example 2 (F6, refuted by a numeric pass). A generated discussion asserts $\pi > 22/7$. Archimedes’ bounds give $223/71 < \pi < 22/7$, so the assertion is false; the two values are $22/7 = 3.142857\dots$ and $\pi = 3.141592\dots$. A single rational-arithmetic check refutes the claim, while the familiarity of $22/7$ as an approximation of π makes the error easy to miss by reading. The refutation is self-contained: no external dataset is required.

Example 3 (F3, refuted by formalisation). A prose proof concludes that every continuous real function is differentiable, via a step that assumes the difference quotient has a limit at every point. Formalisation does not merely disagree with the proof; it fails, because the assumed limit does not exist for $x \mapsto |x|$ at 0. Here the kernel earns its cost – which is exactly why its silence about Examples 1 and 2 deserves to be stated rather than assumed.

5.4 Why residual risk is structural

Proposition 2 (No sound and complete computable verifier for arithmetic). *Let Tr be the set of true sentences of first-order arithmetic. There is no computable procedure M whose accepted set $A(M)$ satisfies both $A(M) \subseteq \text{Tr}$ (soundness) and $A(M) = \text{Tr}$ (completeness).*

Proof sketch. If $A(M) = \text{Tr}$, then Tr would be recursively enumerable, since M enumerates it by dovetailing over all inputs. By Gödel’s first incompleteness theorem, every consistent, recursively axiomatised theory extending Robinson arithmetic is incomplete, and the standard argument shows that Tr is not recursively enumerable. The two claims contradict each other, so no such M exists. $\square$

The practical reading is not defeatism. It says that “certified” must always be read relative to a check set, and that the honest output of a verification pipeline is a tier plus a residual set, never the word *true*. A writing environment that refuses to render a quoted claim without its tier label would have flagged every example in this section.

5.5 Coverage versus cost

Table 4 maps the method families of Section 4 onto the framework. Three patterns are visible. Sampling and voting buy little: they reduce the variance of the answer, not the probability that the argument is wrong. Process supervision buys the inference layer but shares priors with the generator, so its errors correlate with the errors it is meant to catch. Formal proving buys the inference layer outright and is the only family that does so, at a cost that scales with the distance between the informal statement and the available library – a decade of work by twenty-two listed authors for one theorem in the extreme case (Hales et al., 2015). Nothing in the table retires F1, F5, F7, F8, or F10 except an independent audit, which is why the audit sits at the top of the partial order and not at the bottom.

Table 4: Method families mapped to the tier framework. “Partial” marks coverage that is statistical or restricted to a sub-class of claims; the residual set is the taxonomy minus the retired column.

Method family	Tier	Defects retired	Residual risk
Majority vote and self-consistency (Wang et al., 2022)	T1	F6 partial	F1–F5, F7–F10
Outcome reward model (Cobbe et al., 2021)	T1	F6 partial	F1–F5, F7–F10
Process reward model, step supervision (Lightman et al., 2023; Uesato et al., 2022; Wang et al., 2023)	T3	F3, F9 partial	F1, F2, F5, F7, F8, F10
Self-correction with an external corrector (Welleck et al., 2022)	T2/T3	F3 partial, F6 partial	F1, F2, F5, F7, F8, F10
Intrinsic self-correction without external feedback (Huang et al., 2023)	T0/T1	none reliable	all
Retrieval grounding (Lewis et al., 2020)	T2	F7 partial	F1, F3–F6, F8–F10
Autoformalisation plus kernel check (Wu et al., 2022; Jiang et al., 2022; Yang et al., 2023)	T4	F3, F4	F1, F2 detection, F5, F7–F10
Generator plus external critics (LLM-Modulo) (Kambhampati et al., 2024)	T3+T5	F3, F4, F7, F9	F1, F2 detection, F5, F8, F10
Independent reproduction and audit	T5	F1, F2, F5, F7, F8, F10	defects outside the audited set

6 An Audit Protocol

The framework becomes usable only when it leaves the paper, so we state a twelve-item protocol ordered so that mechanical checks come first and judgement-bearing gates come after. Table 5 lists the items together with the defects each one retires. Item 3 is the only item that retires F1, and item 8 is the only item that retires F7.

The protocol is backed by a ledger rather than by good intentions. For each claim we require one row recording the claim identifier, the statement and assumptions verbatim, the tier attained, the residual set, the pointer to the evidence bundle, and the name of the party who signed the fidelity gate (Appendix A). Two properties do the real work. Every claim appears exactly once, so a claim cannot quietly migrate to a stronger tier between drafts; and claims without a row are visible, so absence of evidence is distinguishable from evidence of absence.

7 Limitations

We state the boundaries of what this paper establishes.

1. **One probe, not a controlled measurement.** The framework is a literature-grounded an-

Table 5: The twelve-item audit protocol. “Kind” distinguishes mechanical checks from judgement-bearing gates; the latter are the items that must be signed by an accountable party.

#	Item	Kind	Retires
1	Claim inventory: every claim in the artefact, including captions, abstracts, and code comments, is enumerated	mechanical	—
2	Statement ledger: statement and assumptions recorded verbatim with the intended informal meaning in the author’s words	judgement	prerequisite for F1
3	Fidelity review by a party that did not write the formal statement	judgement	F1, F2
4	Assumption discharge list: every imported lemma recorded with the hypotheses under which it holds	mechanical	F4
5	Kernel record: prover name, version, commit hash, and the exact statement file	mechanical	F3, F4
6	Numeric spot checks with an independent tool, including at least one bound or inequality	mechanical	F6
7	Citation verification against a primary identifier (DOI or archive record), one entry at a time, with the checked identifier recorded	mechanical	F7 partial
8	Contamination check: overlap between the evaluated set and available training data, with the result disclosed either way	judgement	F7, F8
9	Evidence-to-claim linkage: each claim tagged with its tier and residual set	mechanical	—
10	Environment pins for reproduction: versions, seeds, and a runnable entry point	mechanical	F10
11	Adversarial pass: an explicit attempt to refute each load-bearing claim rather than to confirm it	judgement	F5, F9
12	Disclosure of AI involvement and of failed attempts, including abandoned branches	judgement	F9, F10

alytic synthesis. The only measurement in this paper is the fidelity probe of Section 5.2: 22 hand-built pairs, not a random sample, so its proportions describe this checker on this probe set and must not be read as defect rates. We report no inter-annotator agreement and no measured timings; the cost classes in Table 2 remain order-of-magnitude estimates, and a reader who needs rates must collect them.

2. **The fidelity gate is the weakest joint.** Item 3 of the protocol carries the largest share of the residual risk. Section 5.2 shows the gate is partly mechanisable: the surface-visible part of F1 fell to a 24-word checker at zero false-alarm cost, so a machine check narrows the gate rather than closing it. We still did not measure how often independent reviewers agree on fidelity judgements, and a disagreement-prone gate would undermine the tier it is meant to support.
3. **The taxonomy is not proven exhaustive.** F1–F10 are grounded in the literature we cite and in the examples we worked, but new model families and new tool interfaces will produce defect classes we have not named. The framework tolerates this by construction – an unnamed defect simply belongs to the residual set – but the taxonomy itself is not closed.
4. **Proposition 2 is limitative, not constructive.** It explains why residual risk cannot be

driven to zero. It does not tell a practitioner which checks to run, and it says nothing about non-arithmetic domains such as geometry, where diagrammatic evidence plays a role this taxonomy does not model.

5. **Our own citation checking is bounded.** Every bibliographic entry in this paper was checked against a primary identifier at write time, but we did not re-derive the results we attribute to those works.
6. **Automated review is not a substitute for the audit.** Nothing here licenses replacing human sign-off of the fidelity gate with another language model: a model auditing a model shares the failure distribution that motivated the audit.
7. **Scope of the examples.** The three worked examples are didactic and arithmetic or analytic in character. They establish that the defects exist and are detectable at the stated tiers; they are not a sample from which defect frequencies can be estimated.

8 Conclusion

The reliability problem in AI-assisted mathematics is usually framed as a capability problem: models are not yet good enough at proof. That framing hides a second problem which better models do not solve. Verification is layered, the layers are not interchangeable, and the strongest mechanised layer on offer – formal proof – is silent about whether the sentence it certified is the sentence that was meant. We proposed a ten-item failure taxonomy, a six-level tier framework with an explicit coverage profile, two propositions that fix the tier order and explain why residual risk is structural, a twelve-item audit protocol backed by a claim-level ledger, and a reproducible probe that fixes how far the fidelity gate can be mechanised.

The framework reduces to three instructions. Report the tier, never the adjective. Pair the formal tier with an adversarial audit, because they retire disjoint defect classes. Treat “certified” as relative to a check set, and publish the residual set wherever a claim is quoted outside its own paper.

Two lines of work follow. The empirical one is to grow the fidelity probe of Section 5.2 from a hand-built set into a sampled one and to measure inter-reviewer agreement for the gate, the framework’s weakest joint; the probe already shows where that measurement must concentrate, namely on converse and direction errors, which no check over the sentence alone can reach. The constructive one is to build tooling that refuses to render a claim without a tier label, turning the ledger into a property of the writing environment rather than a task performed afterwards.

References

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. In *arXiv preprint arXiv:2110.14168*, 2021.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

- Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, and Julius Berner. Mathematical capabilities of Chat-GPT. In *arXiv preprint arXiv:2301.13867*, 2023.
- Thomas Hales, Mark Adams, Gertrud Bauer, Dat Tat Dang, John Harrison, Truong Le Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Thang Tat Nguyen, Truong Quang Nguyen, Tobias Nipkow, Steven Obua, Joseph Pleso, Jason Rute, Alexey Solovyev, An Hoai Thi Ta, Trung Nam Tran, Diep Thi Trieu, Josef Urban, Ky Khac Vu, and Roland Zumkeller. A formal proof of the Kepler conjecture. In *arXiv preprint arXiv:1501.02155*, 2015.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *arXiv preprint arXiv:2103.03874*, 2021.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *arXiv preprint arXiv:2310.01798*, 2023.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023. arXiv preprint arXiv:2202.03629.
- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *arXiv preprint arXiv:2210.12283*, 2022.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. In *arXiv preprint arXiv:2207.05221*, 2022.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can’t plan, but can help planning in LLM-Modulo frameworks. In *arXiv preprint arXiv:2402.01817*, 2024.
- Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *arXiv preprint arXiv:2205.11491*, 2022.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *arXiv preprint arXiv:2005.11401*, 2020.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *arXiv preprint arXiv:2305.20050*, 2023.

- Haohan Lin, Zhiqing Sun, Sean Welleck, and Yiming Yang. Lean-STaR: Learning to interleave thinking and proving. In *arXiv preprint arXiv:2407.10040*, 2024.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. In *arXiv preprint arXiv:2408.06292*, 2024.
- Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. In *arXiv preprint arXiv:2410.05229*, 2024.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. In *arXiv preprint arXiv:2009.03393*, 2020.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625, 2024. doi: 10.1038/s41586-023-06924-6.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625, 2024. doi: 10.1038/s41586-023-06747-5.
- George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amityay Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the putnam mathematical competition. In *arXiv preprint arXiv:2407.11214*, 2024.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. In *arXiv preprint arXiv:2211.14275*, 2022.
- Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *arXiv preprint arXiv:2312.08935*, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *arXiv preprint arXiv:2203.11171*, 2022.
- Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. In *arXiv preprint arXiv:2211.00053*, 2022.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *arXiv preprint arXiv:2205.12615*, 2022.
- Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. In *arXiv preprint arXiv:2405.14333*, 2024.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *arXiv preprint arXiv:2306.15626*, 2023.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *arXiv preprint arXiv:2109.00110*, 2021.

A Claim-level provenance ledger

Table 6: Ledger template. One row per claim; the fidelity column is signed by a party other than the author of the statement.

Claim ID	Statement and assumptions (verbatim)	Tier	Residual set	Evidence pointer	Fidelity signed by
C-01	<i>(statement)</i>	T4	F1, F5, F7, F8, F10	<i>(path or hash)</i>	<i>(name)</i>
C-02					
C-03					